



AI CODING · CURSOR · SOFTWARE DEVELOPMENT

Cursor AI를 활용한 소프트웨어 개발 기본 과정

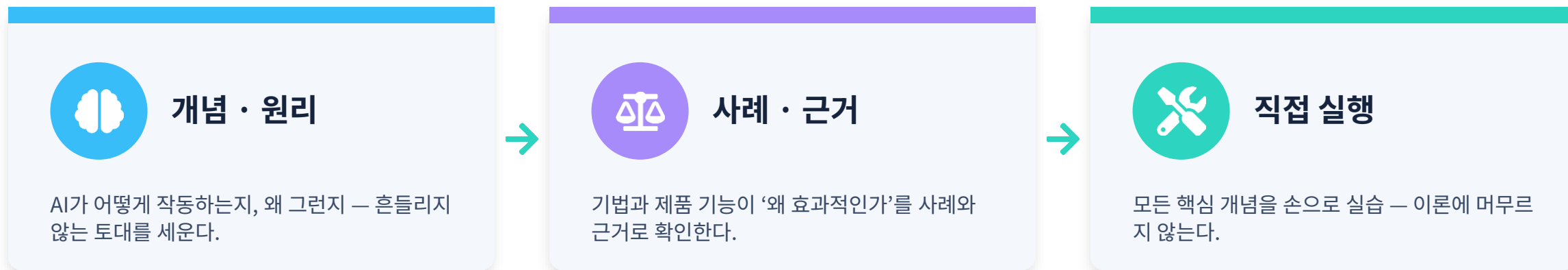
AI 리터러시부터 프롬프트 엔지니어링, 그리고 AI 코딩 에이전트 실전까지 — 8교시 강의 과정

Prepared by **DaeKyeong Kim, Ph.D.**

대상: 학부 고학년 · 대학원 · 실무자 입문 | 구성: 4개 장 · 총 8교시 | 2026년 상반기 기준

강의 개요 — 세 층위를 하나의 흐름으로

본 과정은 'AI를 비판적으로 이해하는 힘'에서 출발해 '프롬프트로 AI를 정확히 부리는 기술'을 거쳐, 마지막에 'AI 코딩 에이전트로 실제 소프트웨어를 만들어 배포하는 경험'까지를 하나의 흐름으로 연결합니다.



구성 원칙

각 장은 (1) 개념과 원리 → (2) 사례·근거 → (3) 실습 순으로 진행됩니다. 강의를 마치면 수강생은 개념·원리·실습의 세 층위를 모두 확보하게 됩니다.

교시별 진행표 · 총 8교시

교시	장	주제	형태
1교시	1장	AI 리터러시 · 생성형 AI · 2026 지형 · LLM · AI 2027	강의·실습
2교시	2장	프롬프트의 원리 · 두 가지 기본 원칙 · 핵심 기법 / RAG · 에이전트 · 반복적 개발 · 실습	강의·실습
3교시	3장	Cursor 입문 · 바이브 코딩 · 자율성 · 핵심 기술 개관	강의·실습
4교시	3장	상호작용(Tab~Agent) · 맥락(@) · Rules · MCP	시연·실습
5교시	3장	Commands · Skills	시연·실습
6교시	3장	Subagents · Hooks	시연·실습
7교시	4장	Cursor로 테트리스 만들기 (단계별 프롬프트)	실습
8교시	4장	GitHub 배포 · 마무리	실습·마무리

선수 지식과 준비물



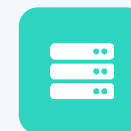
선수 지식

- 브라우저와 기본 파일·폴더 조작 능력
- 프로그래밍 무경험자도 OK — 4장은 모든 단계를 프롬프트로 제공



필요 계정

- ChatGPT 또는 Claude 중 1개 (2장 실습)
- Cursor 계정 + 설치본 (3·4장)
- GitHub 계정 (4장 배포)



실습 환경

- Cursor 1.7+ (Hooks), 가급적 2.4+ (Skills·Subagents)
- 인터넷 연결 · Node.js 권장
- JSON 파싱용 jq (Hooks 실습)



버전 확인

Cursor는 베타 기능이 있습니다. Help → About Cursor (또는 Cmd/Ctrl+Shift+P → 'About')에서 버전이 1.7 이상인지 확인하세요. 강의 직전 공식 문서로 최신 상태를 재확인하는 것을 권장합니다.

1

AI 리터러시와 2026년 인공지능의 지형

1교시 · 무엇을 ‘쓸 줄 아는가’가 아니라 ‘어떻게 읽고 판단하는가’

이 장에서 다루는 것

- AI 리터러시의 정의와 네 기둥
- 딥러닝의 계보와 생성형 AI — CNN · 트랜스포머 · 생성 모델 · LLM
- 2026년 꼭 알아야 할 다섯 가지 흐름
- 유료 · 무료 LLM의 종류와 선택 기준
- ‘AI 2027’ 시나리오와 균형 잡힌 시각

AI 리터러시: 왜 ‘사용법’이 아니라 ‘문해력’인가

AI 리터러시란 AI가 무엇을 할 수 있고 없는지 이해하고, 그 결과를 비판적으로 평가하며, 윤리적·책임 있게 활용·판단하는 역량입니다. 도구를 ‘쓸 줄 아는 것(usage)’과 구분되는 ‘읽고 해석하는 능력(literacy)’이 핵심입니다.



인지 (Know)

AI가 어떻게 학습·작동하는가. 예) LLM은 ‘다음 토큰을 확률적으로 예측’하는 시스템이다.



활용 (Use)

프롬프트·도구·워크플로를 설계해 실제 과업을 해결하는 실천 능력.



평가 (Evaluate)

출력의 사실성·편향·한계를 검증. 환각(hallucination)을 전제로 다루는 습관.



윤리 (Ethics)

저작권·개인정보·책임 소재·노동/사회적 영향에 대한 분별.

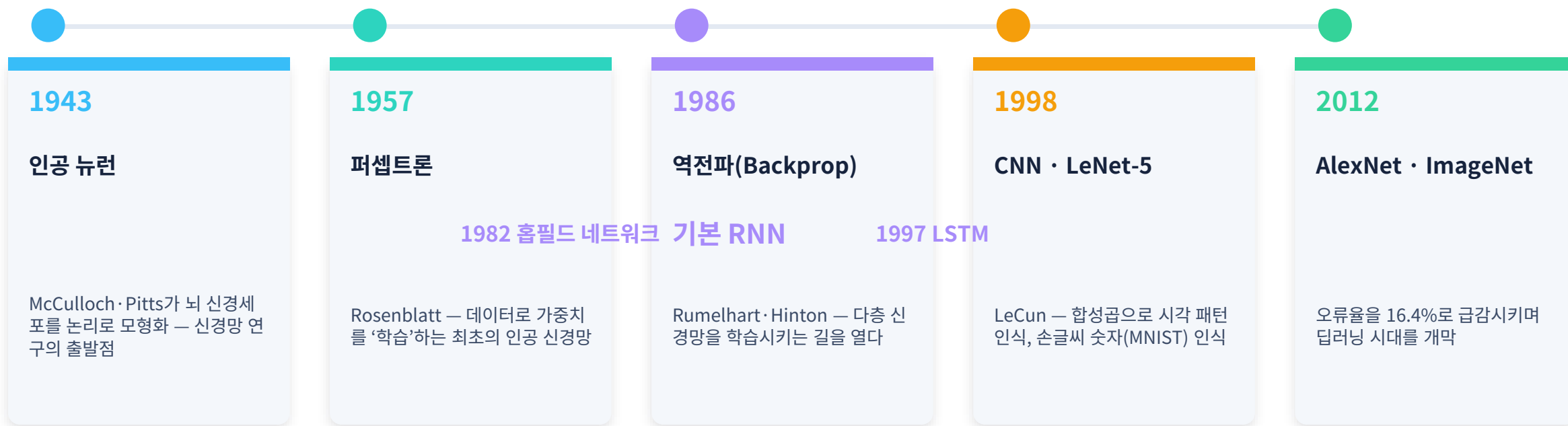


왜 ‘리터러시’인가

인쇄술 이후 글 읽기가 시민의 기본기가 되었듯, 생성형 AI 이후엔 출력을 의심·검증하는 능력이 기본기가 되었다. 사용법은 6개월이면 바뀌지만 문해력은 오래간다.

딥러닝의 계보 — 인공 뉴런에서 트랜스포머까지

오늘의 생성형 AI는 70여 년에 걸친 신경망 연구의 축적 위에 서 있습니다. 핵심 분기점을 짚어 봅니다.



도약의 세 가지 조건

알고리즘만으로는 부족했다. 대규모 데이터(ImageNet)·병렬 연산(GPU)·학습 알고리즘이 맞물리며 2010년대 딥러닝이 폭발했고, 이 흐름이 트랜스포머와 생성형 AI로 이어진다.

CNN과 컴퓨터 비전 — 이미지를 ‘보는’ 신경망

합성곱 신경망(CNN)은 이미지·영상처럼 공간 구조를 가진 데이터를 처리하는 데 특화된 딥러닝 모델입니다. 합성곱 → 비선형 → 풀링 → 완전연결의 흐름으로 특징을 단계적으로 추출합니다.



합성곱 (Convolution)

작은 필터를 이동시키며 이미지의 국소 특징을 추출. 가중치 공유로 효율적이다.



비선형 (ReLU)

음수는 0으로 — 복잡한 패턴을 표현할 수 있게 하는 활성화 함수.



풀링 (Pooling)

영역을 대표값으로 압축 — 차원 축소와 위치 변화에 대한 강건성 (invariance).



완전연결 (FC)

추출된 특징을 모아 최종 분류·판단을 수행하는 계층.



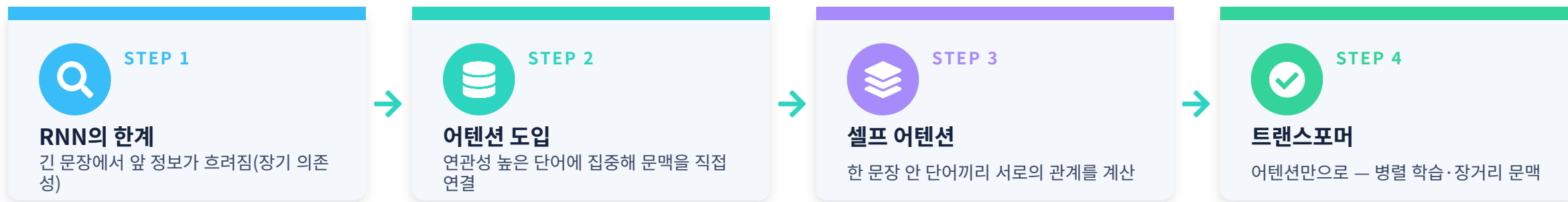
ImageNet이 연 시대

2009년 공개된 1,400만 장 규모의 ImageNet과 ILSVRC 대회가 CNN 경쟁을 촉발했다. 오늘날 분류·검출(Detection)·분할(Segmentation) 등 컴퓨터 비전 과업의 토대가 되었다.

어텐션과 트랜스포머 — 오늘날 LLM의 핵심 구조

어텐션과 트랜스포머는 **현대 LLM이 문맥을 이해하는 방식의 핵심입니다**. 사람의 시각이 중요한 곳에 ‘집중’하듯, 어텐션은 입력에서 중요한 부분에 가중치를 둡니다.

초기엔 순환신경망(RNN)에 어텐션을 덧붙였지만, 2017년 트랜스포머는 순환 구조를 버리고 어텐션만으로 문맥을 처리하는 길을 열었습니다.



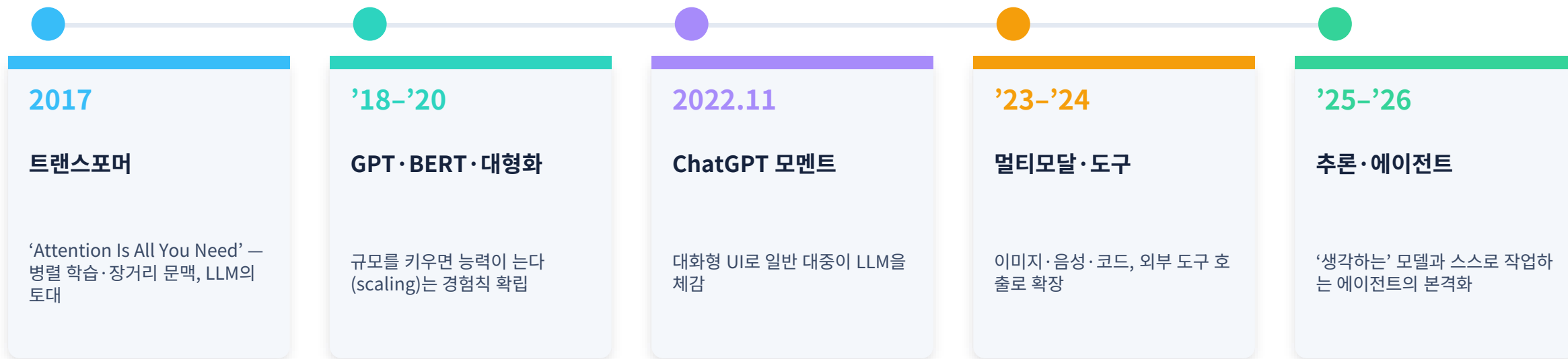
핵심

트랜스포머는 문장 속 단어 쌍의 관계를 한 번에 계산해 병렬 학습과 장거리 문맥 처리를 가능케 했다. GPT·BERT·Claude·Gemini 등 거의 모든 현대 LLM의 근간이다.

분기점 → ‘Attention Is All You Need’(2017)는 NLP의 판도를 바꾼 논문입니다.

생성형 AI의 등장: 트랜스포머에서 추론·에이전트까지

기존 AI가 분류·예측(이 사진은 고양이인가?)에 머물렀다면, 생성형 AI는 창작·작성·합성(고양이를 그려라)을 수행합니다. 핵심 전환점을 따라가 봅시다.



instruction-tuned 모델

오늘의 챗봇은 단순히 다음 단어를 잇는 '기반(base) 모델'이 아니라, 지시를 따르도록 추가 학습(instruction tuning)과 인간 피드백 강화학습(RLHF)을 거친 모델. 이 '지시 추종성'이 프롬프트 엔지니어링의 전제다.

생성 모델의 세 갈래 — VAE · GAN · Diffusion

생성 모델은 데이터의 확률분포를 학습해 ‘새로운’ 데이터를 만듭니다. 대표 계열 셋과 그 응용을 살펴봅니다.



1

VAE · 변분 오토인코더

데이터를 잠재 공간으로 압축·복원. 확률분포로 표현해 다양한 출력을 빠르게 생성.



2

GAN · 적대적 생성망

생성자와 판별자가 경쟁하며 학습. 선명한 고품질 이미지 생성에 강점.



3

Diffusion · 확산 모델

노이즈를 더했다 제거하며 학습. Stable Diffusion · DALL·E의 기반.



4

응용 · 인페인팅/초해상도

비어 있는 영역을 채우고(inpainting), 저해상도를 고해상도로 복원(SR).



5

응용 · 데이터 증강/3D

부족한 학습 데이터를 생성하고, NeRF·3D-GAN으로 장면을 복원.



세 모델의 트레이드오프

어떤 모델도 품질·다양성·속도를 모두 잡진 못한다 (trilemma). GAN은 빠르고 선명, VAE는 빠르고 다양, Diffusion은 고품질이나 느리다.

멀티모달 생성 AI — 텍스트로 무엇이든 만든다

하나의 프롬프트로 이미지·영상·음악·3D까지. 서로 다른 형태(모달리티)를 함께 학습한 모델들이 'Text-to-X' 생성을 가능케 합니다.



Text-to-Image

텍스트로 이미지 생성

"DALL·E · Stable Diffusion"



Text-to-Video

텍스트로 영상 생성

"Sora · Veo 3 · Gen-3 Alpha"



Text-to-Audio

텍스트로 음악·음성 생성

"MusicLM · AudioLM"



Text-to-3D

텍스트로 3D·장면 생성

"DreamFusion · NeRF"



멀티모달 이해

이미지를 보고 설명·대화

"GPT-4o · Gemini · Claude"



산업 응용

여러 센서·데이터의 융합

"자율주행 · 의료 영상 진단"

LLM은 어떻게 작동하는가 — 자연어 처리의 정점

거대 언어 모델(LLM)은 방대한 텍스트·코드로 단어 사이의 통계적 관계를 학습한 자연어 처리(NLP) 모델입니다. 다음 단어를 확률적으로 예측하며 일관된 문장을 생성합니다. 핵심 단계를 따라가 봅시다.



사전 학습 (Pre-training)

인터넷 규모의 텍스트·코드로 ‘다음 단어 예측’을 학습 — 문법과 지식의 토대를 쌓는다



미세 조정 (Fine-tuning)

지시 따르기·특정 작업에 맞게 추가 학습(instruction tuning·RLHF)



문맥 이해 (Self-Attention)

앞뒤 단어의 관계를 고려해 일관되고 자연스러운 텍스트를 생성



작업 적응 & 배포

번역·요약·질의응답 등으로 응용하고 챗봇·서비스로 배포



LLM 계보 한눈에

Eliza(1966) → LSTM(1997) → 트랜스포머(2017) → GPT-3(2020)로 이어진 흐름. 오늘날 ChatGPT·Claude·Gemini·Llama 등 거의 모든 LLM이 트랜스포머의 셀프 어텐션을 근간으로 한다.

2026년, 꼭 알아야 할 다섯 가지 흐름

모델 이름을 외우기보다 ‘흐름’을 잡는 편이 유효합니다. 개별 버전은 분기마다 바뀌지만 흐름은 더 오래 갑니다.



1

추론(Reasoning) 보편화

즉답 대신 단계를 밟아 ‘생각한 뒤’ 답한다. 수학·코딩·분석 정확도 ↑



2

에이전트로의 이동

챗봇 → 도구를 쓰고 여러 단계를 스스로 수행하는 행위자로 무게중심 이동



3

멀티모달의 일상화

텍스트·이미지·음성·문서·영상이 한 모델에서, ‘화면을 보고 조작’까지



4

컨텍스트 확장·메모리

수십만~수백만 토큰의 대형 컨텍스트와 대화 간 ‘기억’이 경쟁 축



5

비용·효율의 분화

최상위 ‘프런티어’ 모델과 값싸고 빠른 경량·온디바이스 모델로 분화



흔한 오해

‘가장 비싼 최신 모델이 항상 정답’이 아니다. 작업 난이도에 모델을 맞추는 것(easy→경량, hard→프런티어)이 비용·속도·품질의 균형을 만든다.

주요 LLM 지형 (2026 상반기)

버전 표기는 빠르게 바뀝니다. ‘공급자 – 성격 – 접근성’ 중심으로 구조화해 기억하세요.

공급자 / 제품	대표 모델 패밀리	성격	접근성
OpenAI — ChatGPT	GPT-5 계열	범용 · 생태계 · 음성/이미지	무료 + 유료 · API
Anthropic — Claude	Opus/Sonnet/Haiku 계열	코딩 · 긴 작업 · 지시 준수	무료 + 유료 · API
Google — Gemini	Gemini 3 계열	멀티모달 · 워크스페이스 · 가성비	무료 + 유료 · API
xAI — Grok	Grok 4 계열	실시간 소셜 데이터 연동	무료 일부 + 유료
DeepSeek	V3 / R1 계열	고성능 · 저비용, 오픈웨이트 포함	무료/오픈 + API
오픈웨이트	Llama · Qwen · DeepSeek	자체 서버/로컬, 데이터 통제	가중치 무료(직접 호스팅)

무료 · 유료 · API · 오픈웨이트, 그리고 선택 기준



무료 티어

최신 모델 일부를 제한된 사용량으로 체험. 학습·가벼운 작업에 충분.



유료 구독

상위 모델·더 큰 사용량·고급 기능(심화 추론·에이전트·메모리 등).



API

내 앱에 모델을 ‘부품’으로 끼워 넣는 방식. 토큰 단위 종량 과금.



오픈웨이트

가중치를 받아 직접 구동. 데이터 통제에 유리, 운영 부담은 본인 몫.

선택 기준 체크리스트



작업 성격 코딩 / 글쓰기 / 멀티모달 / 실시간 정보 중 무엇인가?



민감도 사내·개인정보가 모델로 전송돼도 되는가? (아니면 오픈웨이트/엔터프라이즈)



비용·사용량 한 달 사용량과 예산. 모든 작업에 비싼 모델이 필요하진 않다.



생태계 이미 쓰는 도구(문서·코드·검색)와의 연동성은?

‘AI 2027’: 미래를 구체적으로 상상하기

‘AI 2027’은 전 OpenAI 연구자 대니얼 코코타일로가 이끄는 AI Futures Project가 2025년 발표한 ‘시나리오 예측’ 보고서입니다. 추상적 경고 대신 월(月) 단위의 구체적 이야기로 위험을 ‘눈에 보이게’ 만든 점이 특징입니다.



출발

2025 시작 → 2026 에이전트 부상 → 2027 초 코딩 대부분 자동화 → 2027 말 능력 폭발



핵심 동력

재귀적 자기개선 — AI가 ‘AI 연구 자체’를 가속하며 능력이 복리처럼 증가



세 경쟁 압력

기업·국가(지정학)·정렬(alignment) 문제가 서로를 부추겨 ‘안전보다 속도’로



갈림길(분기)

2027 중반: ‘속도를 늦추는 길’ vs ‘질주하는 길’ — 통제 실패 또는 긍정적 결말



균형 잡힌 결론 — ‘예언’이 아니라 ‘사고 실험’

정책·대중 토론을 촉발한 교육적 가치가 크지만, 핵심 가정(특히 재귀적 자기개선 속도)이 과장됐을 수 있고 ‘파국 프레임’이 공포를 키운다는 비판도 있다. 반대로 ‘AI as Normal Technology’처럼 점진적 변화를 강조하는 관점도 있다. → 어떤 예측도 가정의 산물이며, 가정을 의심하는 것이 곧 AI 리터러시다.

핵심 정리



AI 리터러시 = 인지 · 활용 · 평가 · 윤리의 종합. ‘사용법’보다 ‘문해력’.



생성형 AI의 진화 트랜스포머 → 대형화 → ChatGPT 모멘트 → 멀티모달 → 추론/에이전트.



2026년의 흐름 추론 · 에이전트 · 멀티모달 · 대형 컨텍스트/메모리 · 비용 분화.



LLM 선택 ‘공급자-성격-접근성(무료/유료/API/오픈웨이트)’으로 구조화해 고른다.



AI 2027 가정을 의심하며 읽어야 할 ‘구체적 사고 실험’.

2

프롬프트 엔지니어링의 원리와 실제

2교시 · 명확한 의사소통을 ‘모델이 알아듣는 형태’로 정교화하는 공학

이 장에서 다루는 것

- 프롬프트가 작동하는 원리
- 두 가지 기본 원칙과 실전 ‘다섯 가지 원리’
- 핵심 기법 — zero/few-shot · CoT · 역할 · 형식 · 체이닝
- 한 단계 위: RAG와 에이전트
- 반복적 개발 · 평가와 따라 하는 실습

프롬프트란 무엇이며 왜 중요한가

프롬프트(prompt)는 모델에게 주는 입력 — ‘지시·질문·맥락의 묶음’입니다. LLM은 근본적으로 ‘지금까지의 텍스트 다음에 올 토큰을 확률적으로 예측’하는 시스템이므로, **입력이 출력의 분포를 결정**합니다. 같은 모델이라도 프롬프트가 출력 품질을 좌우하는 이유입니다.



기반(base) 모델

그저 ‘다음 단어를 잇는’ 모델. 지시를 따른다는 보장이 없다. 자유 연상에 가깝다.



instruction-tuned 모델

우리가 쓰는 챗봇. ‘지시를 따르도록’ 추가 학습됨. ‘무엇을·어떻게·어떤 형식으로’ 명확히 지시할수록 정확히 따라온다.



핵심 관점

프롬프트 엔지니어링은 ‘마법 주문 찾기’가 아니다. 명확한 의사소통을 모델이 알아들을 수 있는 형태로 정교화하는 공학적 반복 과정이다. ‘완벽한 프롬프트’를 인터넷에서 찾으려 하지 말고, 직접 다듬어라.

원칙 1 — 명확하고 구체적인 지시를 써라



‘명확함’ ≠ ‘짧음’

더 긴 프롬프트가 더 많은 맥락을 줘서 더 명확할 때가 많다. 다음 네 가지 전술이 자주 쓰인다.



전술 1 구분자(delimiter) 사용

처리 대상을 따옴표·백틱·꺾쇠·태그로 감싸 ‘데이터의 경계’를 분명히. 프롬프트 인젝션 방지에도 도움.



전술 2 구조화된 출력 요구

JSON·표·머리글 등 원하는 형식을 명시. 예) ‘key book_id, title...를 가진 JSON으로 답하라’.



전술 3 조건 확인 요청

전제가 충족되는지 먼저 검사. 예) ‘단계가 있으면 번호를, 없으면 「단계 없음」이라고만’.



전술 4 모범 예시 제공 (few-shot)

원하는 입출력 예를 한두 개 보여 주면 형식과 톤을 정확히 모사한다.

원칙 2 — 모델에게 ‘생각할 시간’을 줘라

사람도 어려운 문제를 즉답하면 틀리듯, 모델도 곧장 결론부터 내면 오류가 늘어납니다. 추론 과정을 펼치게 하면 정확도가 오릅니다.



전술 1 · 단계 지정

과업을 하위 단계의 연속으로 쪼개 ‘1 → 2 → 3’ 순서로 수행하게 한다.



전술 2 · 결론 전에 스스로 풀게

‘학생 풀이가 맞는지 평가하라’가 아니라 ‘먼저 네가 직접 풀고, 그 다음 비교하라’. 답을 미리 정해두고 합리화하는 것을 막는다.



환각(hallucination) 경고

모델은 잘 모르는 주제에서도 그럴듯하게 지어낸다. 사실이 중요한 작업에서는 ① 근거/출처를 함께 요구하고, ② ‘모르면 모른다고 답하라’를 명시하며, ③ 결과를 사람이 검증한다. 프롬프트가 좋아도 검증 습관은 별개다.

프롬프트의 다섯 가지 원리 — 실패를 푸는 처방

Andrew Ng의 두 원칙을 실전 체크리스트로 확장한 프레임워크. 흔한 실패 다섯 가지에 각각 대응하는 원리를 적용합니다.



1. Give Direction

방향 제시 ↔ 모호한 지시

“타깃·스타일·길이를 구체적으로”



2. Specify Format

형식 지정 ↔ 형식 없는 출력

“JSON·표·목록 등 형식을 명시”



3. Provide Examples

예시 제공 ↔ 누락된 예시

“원하는 입출력 예시를 보여주기”



4. Evaluate Quality

품질 평가 ↔ 막연한 평가

“정량 기준(Rubric)으로 자체 채점”



5. Divide Labor

작업 분할 ↔ 한 번에 다 하기

“복잡한 일은 단계로 쪼개 연결”



핵심 한 줄

다섯 처방의 합

“방향+형식+예시+평가+분할”

5대 원리 실전 — Bad에서 Good으로 (운동화 작명)

원리	Bad · 흔한 실패	Good · 개선된 프롬프트
방향 제시	“운동화 이름 지어줘” → FastShoes (뻘함)	“20대 여성 · 2음절 신조어로” → Aura · Velo
형식 지정	“이름 3개 추천” → “물론이죠! 1...” (파싱 불가)	“서론 없이 리스트로만” → [“Zoomies”, “AirStep”]
예시 제공	“창의적 이름” → RunningPro (평범)	“Nike(승리의 여신)처럼 상징적으로” → Kiros
품질 평가	“느낌이 별로네, 다시” (막연)	“발음 · 상표권 · 도메인 3기준 5점 채점과 함께”
작업 분할	“이름 · 로고 · 슬로건 한 번에” (얕은 결과)	①키워드 → ②조합 → ③슬로건, 단계로 나뉘

형식 지정의 힘 — AI는 ‘만능 번역기’다

출력 형식을 명시하면 결과를 바로 코드·문서로 활용할 수 있습니다. LLM은 언어뿐 아니라 ‘구조’도 번역합니다.



전술 1 · 구조화된 출력 강제

‘서론·번호 없이 JSON 리스트로만 출력’처럼 형식을 못박으면 프로그램이 결과를 바로 파싱할 수 있다.



전술 2 · 자주 쓰는 데이터 형식

JSON(키-값 구조) · CSV(표·대량 데이터) · YAML(설정·높은 가독성). 모두 텍스트 기반이라 ChatGPT가 직접 생성·변환한다.



‘만능 번역기’의 의미

LLM은 언어 간 번역(한↔영)뿐 아니라 형식 간 번역(JSON↔YAML)과 자연어↔코드 변환까지 수행한다. ‘무엇을, 어떤 형식으로’를 함께 지정하면 후속 자동화가 쉬워진다.

프롬프트의 유형 분류

목적과 구조에 따라 프롬프트를 여섯 가지로 나눠 보면, 어떤 상황에 어떤 형태를 써야 할지 감을 잡기 쉽습니다.



질문형

정보 탐색 · 설명 요청

“머신러닝이란 무엇인가요?”



지시형

특정 작업 수행 요청

“Python으로 정렬 함수를 작성해줘”



역할 부여형

페르소나 설정

“너는 시니어 백엔드 개발자야”



분석형

데이터 · 코드 분석

“이 코드의 시간 복잡도를 분석해줘”



창작형

새로운 콘텐츠 생성

“개발 원칙 5가지를 만들어줘”



대화형

멀티턴 상호작용

“이전 답변을 바탕으로 더 구체화해줘”

P-C-T-F와 7단계 프레임워크



응답 품질의 상당 부분은 프롬프트 설계에서 갈린다

같은 모델이라도 어떻게 구성하느냐에 따라 결과가 크게 달라집니다 — 구조화가 곧 품질입니다.

P-C-T-F 패턴 — 가장 기본이 되는 4요소



Persona · 역할 (목표)부여

너는 시니어 백엔드 개발자다.
너의 목표는 보안성, 테스트 가능성, 유지보수성을 갖춘 백엔드 기능을 설계하는 것이다.



Context · 배경 상황

“현재 주문 시스템을 설계 중”



Task · 수행 과제

“RESTful API 설계안 3가지 제안”



Format · 출력 형식

“각각 장단점을 표로 정리”

7단계 프레임워크 — 정밀 과업을 위한 확장

1

Role & Goal 기획 전문가, 목표: 요구사항 분석

2

Context & Input 현재 신규 쇼핑몰 프로젝트

3

Constraints 300자 이내, 안전 규칙 준수

4

Output Format 마크다운 표 형식으로

5

Example 예시: 문제→해결 3가지

6

CoT “Step by step으로 생각해”

7

Feedback Loop 응답 검토 후 재질문

핵심 프롬프트 기법 일곱 가지

기법	한 줄 설명	쓰는 상황
Zero-shot	예시 없이 지시만으로 수행	단순·일반적 과업
Few-shot	입출력 예시 1~수 개를 함께 제시	형식·톤·라벨을 정확히 모사할 때
Chain-of-Thought	‘단계적으로 생각하라’로 추론 유도	수학·논리·다단계 추론
역할 부여 (Persona)	‘너는 ~전문가다’로 관점 고정	전문 톤·도메인 지식 유도
출력 형식 지정	JSON·표·길이·언어를 명시	후속 처리·일관성이 필요할 때
프롬프트 체이닝	큰 작업을 여러 프롬프트로 분할·연결	복잡한 파이프라인
자기일관성	여러 번 추론해 다수결	정확도가 중요한 추론

고급 프롬프트 전략 — ICL · CoT · ReAct



In-Context Learning

예시로 가르치기

- 학습 없이 예시만으로 동작 유도
- Zero-shot: 예시 없이 지시만
- One-shot: 예시 1개
- Few-shot: 예시 3~5개 → 정확도 ↑

```
# Few-shot 예시
입력: 고객이 화남 → 부정
입력: 배송이 빠름 → 긍정
입력: 상품이 마음에 듭 → ?
```



Chain-of-Thought

단계별 사고 유도

- “단계별로 생각해줘” 한 줄로 효과
- 복잡한 수학·논리에 강력
- Zero-shot CoT: think step by step
- Few-shot CoT: 풀이 과정 포함 예시

```
# CoT 적용 예
Q: 3명이 6일이면 완료.
  2명이면 며칠?
Step1: 3×6 = 18인일
Step2: 18÷2 = 9일
```



ReAct

추론 + 행동 결합

- Reasoning(추론) + Acting(도구 호출)
- LLM 에이전트의 핵심 패턴
- Thought→Action→Observation 반복
- Cursor·LangChain 내부에서 사용

```
Thought: 파일 목록 확인
Action: list_files("./src")
Obs: ["main.py", "utils.py"]
Action: read_file("main.py")
```

Few-shot & Chain-of-Thought 실전 예시

Few-shot — 예시 형식을 그대로 모사

다음 형식을 따라 감정을 분류하라.

리뷰: "배송이 너무 느렸어요." → 부정

리뷰: "가격 대비 만족합니다." → 긍정

리뷰: "포장은 좋았는데 기대 이하." → 중립

리뷰: "재구매 의사 100%입니다!" →

CoT — 단계 풀이 뒤 결론만 별도 형식으로

문제: 사과 10개를 샀다. 이웃 2개, 수리공 2개를 줬다. 이후 5개를 더 사서 1개를 먹었다. 지금 사과는 몇 개인가?

지시: 먼저 단계별 계산 과정을 보여라. 그 다음 마지막 줄에 "정답: N개" 형식으로만 결론을 적어라.



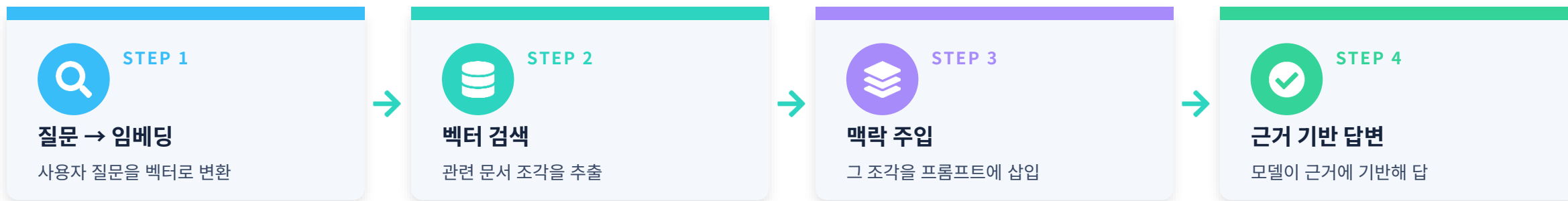
Zero-shot CoT

복잡한 추론에서는 ‘단계적으로 생각하라(Let’s think step by step)’ 한 줄을 덧붙이는 것만으로도 정확도가 오른다. 단계 풀이를 펼치게 한 뒤 마지막에 결론만 별도 형식으로 요구하면 검증도 쉬워진다.

RAG — 검색 증강 생성

프롬프트만으로는 두 한계가 있습니다: ① 학습 시점 이후·사내 비공개 자료를 모른다, ② 스스로 외부 행동(검색·실행)을 못 한다. 이를 메우는 것이 RAG와 에이전트입니다.

RAG는 ‘질문에 관련된 문서를 외부 지식원에서 검색해 프롬프트에 끼워 넣은 뒤 답을 생성’하는 방식. 재학습 없이 최신·비공개 지식을 다루고, 출처를 제시해 환각을 줄입니다.



장점

재학습 불필요 · 지식 갱신 용이 · 출처 제시로 신뢰성 ↑. 지식 집약적·도메인 특화 과업의 표준 패턴.

연결 고리 → 3장에서 다룰 Cursor의 @Docs/@Web이 바로 이 ‘RAG적 맥락 주입’에 해당합니다.

에이전트 — 도구를 쓰고 여러 단계를 스스로

에이전트는 모델이 ‘도구(검색·코드 실행·API 호출)를 스스로 골라 쓰며, 관찰 → 사고 → 행동을 반복’해 목표를 달성하는 구조입니다.

ReAct 루프



함수 호출 (function calling)

모델이 정해진 도구를 호출하게 만드는 핵심 메커니즘.

연결 고리 — 3장 Cursor



Cursor의 ‘Agent 모드’가 바로 이 에이전트 개념의 구체적 구현이다. 2장의 추상 개념(RAG·에이전트)이 3장에서 실제 제품 기능으로 등장한다 — 이 다리를 명시적으로 짚으면 강의 흐름이 매끄럽다.

반복적 개발과 평가

좋은 프롬프트는 한 번에 나오지 않습니다. ‘초안 → 결과 관찰 → 실패 원인 분석 → 프롬프트 수정’의 반복이 정석 — 머신러닝의 ‘아이디어 → 구현 → 실험 → 개선’ 루프와 동일합니다.



→ 이 루프는 4장 테트리스 실습에서 그대로 적용됩니다.

실습 — 구조화된 프롬프트 프레임워크



P-C-T-F 구조

- Purpose 목적
- Context 맥락
- Task 작업
- Format 출력 형식

‘로그인 기능 만들어줘’ → 네 영역으로 명확화



7-Step Framework

- 문제 인식 → 핵심 개념
- 불변 조건 → 유스케이스
- 입출력 계약 → 테스트
- 확장성·리스크 분석

이커머스 장바구니 시스템 설계 실습



Few-Shot 설계

- 예시 3개 (정상/실패 포함)
- 입력 조건 명확화
- 반환 형식 일관성
- 실제 요청 1개 작성

데이터 유효성 검증 함수 생성 유도



불변 조건(Invariant)이란?

‘코드가 어떻게 바뀌어도 절대 깨지면 안 되는 규칙.’ 예) 수량은 1 이상 · 동일 상품 중복 금지 · 총액 = 항목 합계. 테스트로 항상 검증할 수 있어야 하고, 위반되는 순간이 곧 버그다.

핵심 정리



프롬프트 = 입력 출력 분포를 정하는 입력. 우리가 쓰는 모델은 '지시를 따르도록' 학습됨.



두 원칙 명확·구체적 지시 / 생각할 시간 주기 — 거의 모든 기법이 이 둘의 변주.



핵심 기법 zero/few-shot · CoT · 역할·형식 지정 · 체이닝 · 자기일관성.



RAG & 에이전트 외부 지식 주입 / 도구 사용·다단계 수행 — 3장 Cursor에서 실물로 만난다.



반복 & 검증 프롬프트는 반복으로 완성되며, 환각은 검증으로 다스린다.

3

Cursor AI 핵심 기술과 Hooks 실전

3~6교시 · 2장의 '에이전트 · RAG · 프롬프트'가 실제 제품 기능으로

이 장에서 다루는 것

- Cursor란 무엇인가 — 자율성의 슬라이더
- 핵심 기술 10가지 개관
- 상호작용: Tab · Cmd+K · Ask · Agent
- 맥락 주입(@심볼) · Rules · MCP
- Commands · Skills · Subagents
- Hooks — 에이전트 실행 루프 통제

프롬프트 기반 코딩의 진화

‘코드를 직접 쓰는 일’에서 ‘의도를 설계하고 검증하는 일’로 — 개발자의 역할 자체가 바뀌고 있습니다.

~2023



전통적 코딩

- 모든 코드를 수작업으로 작성
- 개발자가 직접 문법·API 암기
- 도구는 자동완성 수준



2024



AI 보조 코딩

- GitHub Copilot · Tabnine 등
- AI가 부분적으로 코드 보완
- 사람이 흐름을 주도



2025~



AI 에이전트 코딩

- Cursor · Cline 등
- AI가 코드 전반을 자동 생성
- 개발자 = 설계자·검증자

3.1 WHAT IS CURSOR

기존 개발 vs 바이트 코딩



기존 개발 방식



직접 코드 작성

한 줄씩 타이핑 · 문서 검색의 반복



수동 디버깅

에러 → 검색 → 원인 추론 → 수정 반복



파일별 편집

파일을 열고 닫으며 컨텍스트 전환



긴 피드백 루프

작성 → 빌드 → 테스트 → 수정 사이클이 느림



지식 검색

API 문서 · 예제를 직접 찾는 시간 소모



바이트 코딩 방식



의도 중심 지시

“로그인 기능 만들어줘” → AI가 자동 생성 · 수정



AI 자율 디버깅

에러 발생 시 원인 분석 → 수정 → 검증 자동



프로젝트 전체 이해

코드베이스 파악 후 관련 파일 동시 수정



즉각적 피드백

지시 → 생성 → 실행 → 확인까지 수분 내 완료



검증 중심 역할

사람은 ‘무엇을’, AI는 ‘어떻게’를 담당

3.1 WHAT IS CURSOR

주요 AI 코딩 에이전트 비교

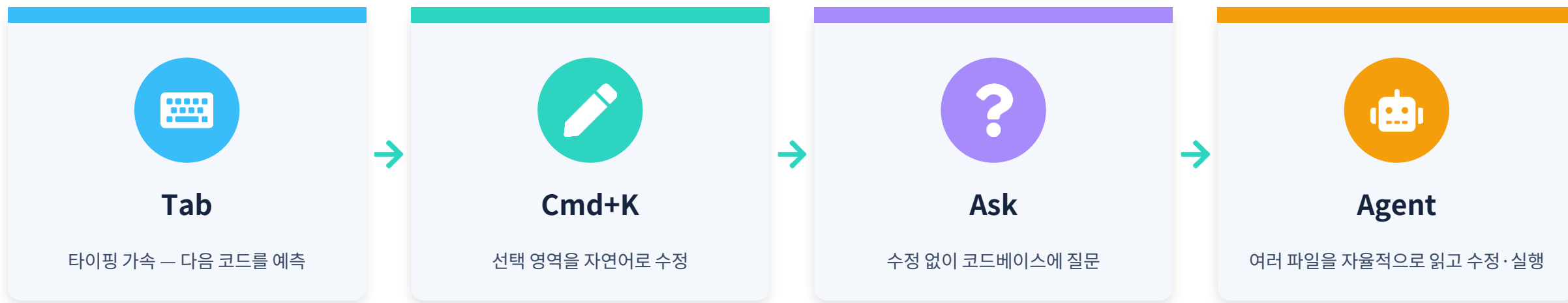
도구	특징	강점	가격
Cursor AI	VS Code 기반 IDE · 멀티파일 편집	Agent/Plan/Ask 3모드 · .cursorrules	Free / \$20~
GitHub Copilot	GitHub 공식 통합 · 전 IDE 지원	기업 보안 강함 · 대형 레포 최적화	\$10~\$19/월
Cline	오픈소스 · VS Code 확장	직접 API 사용 가능 · 커뮤니티 인기	Free (API 비용)
Amazon Q	AWS 생태계 최적화	AWS 서비스 연동 · 기업용	\$19/월
Claude (Anthropic)	Artifacts · Projects 기능	긴 컨텍스트 · 문서 작업 특화	Free / \$20~
Gemini CLI	Google · 터미널 기반	1M 토큰 컨텍스트 · 넉넉한 무료 티어	무료 티어
OpenAI Codex CLI	OpenAI · 터미널 기반	함수 단위 완성 강점 · Copilot 연계	유료 (API)

※ 가격·기능은 2026년 상반기 기준이며, 빠르게 변동될 수 있습니다.

Cursor란 무엇인가 — 자율성의 슬라이더

Cursor는 VS Code 기반의 'AI 코딩 에디터'입니다. 자동완성부터 자연어 편집, 여러 파일을 스스로 읽고 고치고 실행하는 자율 에이전트까지 — 사람이 통제권을 얼마나 위임할지 단계적으로 선택할 수 있다는 점이 핵심입니다.

자율성 ↑ (AI에게 주는 권한이 커진다)



핵심 기술 10가지 + Hooks 개관

- **1**
Tab
멀티라인 코드 예측
- **2**
Cmd/Ctrl+K
인라인 자연어 편집
- **3**
Ask
코드 수정 없이 질문
- **4**
Agent
자율적 읽기·수정·실행
- **5**
Context & @심볼
맥락 주입 (RAG적)
- **6**
Rules
AI 행동 규칙
- **7**
MCP
외부 도구 표준 연결
- **8**
Commands
저장된 슬래시 프롬프트
- **9**
Skills
스크립트 포함 매뉴얼
- **10**
Subagents
격리·병렬 전문 에이전트



+ Hooks 에이전트 실행 루프의 정해진 단계 전후에 사용자 스크립트를 실행 — ‘맥락(Rules)’이 아니라 ‘실행(Runtime)’을 통제하는 6교시 핵심 주제.

Tab · Cmd+K — 예측과 명령



Tab — 코드 자동완성

입력 중인 코드의 다음 줄(들)을 예측해 회색으로 미리 보여 줌. 여러 줄·함수 전체·리팩터링 방향까지 제안, Tab으로 수락.

```
// 사용자 배열에서 이메일만 추출
function extractEmails(users) {
  return users.|
  // → map(u => u.email) 회색 제안
}
```

연쇄 편집: *users*→*members*로 바꾸면 다른 사용처도 따라 고치라 제안, Tab으로 점프.



Cmd/Ctrl+K — 인라인 편집

코드를 선택한 뒤 자연어 지시로 그 자리에서 수정·생성. 채팅창 없이 에디터 안에서 바로 diff를 보고 수락/거절.

- 이 함수에 JSDoc 주석을 추가해줘
- try/catch로 에러 처리를 감싸줘
- 이 for문을 map으로 바꿔줘
- 입력이 배열이 아니면 빈 배열 반환하도록 방어 코드 추가

Tab(예측) vs Cmd+K(명령)의 차이를 비교 시연하면 이해가 빠릅니다.

Ask · Agent — 탐색과 자율 실행



Ask — 코드베이스 질문 모드

코드를 수정하지 않고 질문에만 답하는 ‘읽기 전용’ 모드. 처음 보는 코드/레거시 파악, 버그 원인 탐색, 수정 전 영향 범위 확인에 적합.

안전하게 탐색하다 확신이 서면 Agent로 전환.

@src 폴더 전체로 데이터 흐름을 설명해줘



Agent — 자율 에이전트 (Composer)

여러 파일을 스스로 읽고 코드를 수정하고 터미널 명령을 실행·검증하며 목표를 향해 반복. Hooks가 개입하는 ‘에이전트 루프’가 바로 이 모드.

```
extractEmails/isValidEmail을 email.js로  
분리하고 테스트를 작성·실행해 통과 확인해줘
```

자율성 슬라이더



Tab → Cmd+K → Ask → Agent로 갈수록 AI에게 주는 자율권이 커진다. Agent가 터미널 명령(예: npm test)을 실행할 때는 승인 요청이 뜨고, 변경 사항은 diff로 검토 후 일괄 수락/거절한다. ‘얼마나 위임할지’를 사람이 단계적으로 통제하는 것이 핵심.

맥락 주입(@심볼)과 Rules

Context & @심볼 — 무엇을 보고 답할지 지정

@심볼	의미
@파일명 / @폴더명	특정 파일·폴더를 맥락에 포함
@Code	특정 심볼(함수/클래스) 참조
@Docs	등록된 외부 문서(라이브러리 공식문서)
@Web	실시간 웹 검색 결과
@Git	커밋 / diff 등 git 정보

@Docs / @Web = 2장의 'RAG적 맥락 주입'의 실제 구현



Rules — AI 행동 규칙

프로젝트/사용자 단위로 AI가 '항상 따라야 할' 규칙을 정의. 코딩 컨벤션·금지 라이브러리·응답 언어 등을 고정. `.cursor/rules/*.mdc` 파일 사용.

- 모든 주석·커밋은 한국어로
- JS는 ES Modules 문법만, 함수엔 JSDoc

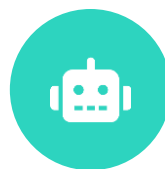


시험에 자주 나오는 핵심

Rules는 '맥락(Context)을 형성'하고, 뒤에 나올 Hooks는 '실행 동작(Runtime)을 통제'한다. 둘의 역할 차이를 분명히 구분하라. (AI가 헛다리 짚는 대부분의 원인은 맥락 부족 — @심볼로 맥락을 좁히는 습관을 들이자.)

MCP — 외부 도구·데이터 연결

외부 도구·데이터 소스(DB, GitHub, Slack, 브라우저, 사내 API 등)를 표준 프로토콜로 AI에 연결합니다. MCP 서버를 등록하면 Agent가 그 서버의 도구를 호출할 수 있습니다.



Cursor Agent



GitHub



DB / 사내 API



Sequential Thinking



Context7 / 검색

설정: Cursor Settings → MCP → 서버 추가(JSON/UI). 예) Smithery 가입 후 검색·연동, GitHub은 PAT(개인 액세스 토큰) 발급 후 등록.



보안

MCP는 'AI의 손발을 확장'하는 기능. 신뢰할 수 있는 서버만 연결하고, 토큰(PAT)은 안전하게 보관하라.

Commands · Skills · Subagents — 무엇을, 어떻게, 어디서



Commands

저장된 슬래시 프롬프트

자주 쓰는 프롬프트를 .md로 저장해 /이름으로 호출. 셋 중 가장 단순 — 분기·스크립트 없음.

호출 수동 (/이름)



Skills

다단계 작업 매뉴얼

여러 단계·규칙·실행 스크립트까지 묶은 매뉴얼 (SKILL.md). Agent가 상황을 보고 스스로 호출.

호출 자동 + 수동



Subagents

격리·병렬 전문 에이전트

메인 Agent가 특정 작업을 위임. 자기만의 격리된 컨텍스트에서 돌고 병렬 실행 가능.

호출 메인에 위임



빠른 판단 흐름 (Rules 포함)

항상 지켜야 하는 규칙인가? → Rules · 그냥 자주 쓰는 프롬프트인가? → Commands
단계가 많고 스크립트도 필요한가? → Skills · 별도 컨텍스트/병렬로 돌려야 하는가? → Subagents

핵심 가치: ‘만드는 에이전트’와 ‘검증하는 에이전트’를 나누면(생성/검증 분리) 규칙 준수율이 올라간다.

Hooks — 에이전트 실행 루프 통제 (베타, 1.7+)

Hooks는 에이전트 실행 루프의 정해진 단계 전후에 사용자 스크립트를 실행하는 장치입니다. 스크립트는 stdin으로 JSON을 받고 stdout으로 JSON을 돌려주며 Cursor와 양방향 통신합니다.

 포맷터 자동 실행

 위험 명령 차단

 비밀키·PII 마스킹

 감사 로그 기록

 컨텍스트 자동 주입

주요 Agent 이벤트 (라이프사이클)

이벤트	시점	차단/변형
beforeShellExecution	셸 명령 실행 직전	allow / deny / ask
beforeMCPExecution	MCP 도구 호출 전	차단 가능
beforeReadFile	파일 읽기 직전	읽기 차단(민감파일 보호)
afterFileEdit	파일 편집 후	관찰(포맷터 실행)
subagentStart / Stop	서브에이전트 시작/종료	시작:차단 · 종료:후속메시지
stop	에이전트 루프 종료	후속 메시지 자동 제출

3.7 HOOKS (2/2)

설정 구조 · 입출력 규약 · 실습 예제

hooks.json 기본 구조

```
{ "version": 1, "hooks": {  
  "afterFileEdit": [{"command": "format.sh"}],  
  "beforeShellExecution": [{  
    "command": "block-dangerous.sh",  
    "matcher": "rm|sudo|git",  
    "timeout": 10, "failClosed": true  
  }]} }
```

출력(stdout) — 차단/허용

```
{  
  "permission": "allow",  
  // "allow" | "deny" | "ask"  
  "user_message": "사용자 메시지",  
  "agent_message": "에이전트 메시지"  
}  
// exit 0=성공 · 2=차단
```



우선순위 Enterprise → Team → Project → User

프로젝트 혹은 신뢰된(trusted) 워크스페이스에서만 실행됨.



failClosed: true 보안 혹은 권장

혹이 실패하면(크래시/타임아웃) 동작을 차단. 기본은 fail-open(통과).



실습 흐름: 관찰 → 차단 → 변형

① 감사 로그(afterFileEdit) — 가장 안전한 첫걸음, 혹은 실제로 발화하는지 눈으로 확인. ② 위험 명령 차단(beforeShellExecution + matcher "rm\s+-rf" + failClosed) — 'rm -rf'는 막고 'ls'는 통과. 난이도를 단계적으로 올린다.

핵심 정리



자율성 슬라이더 Tab → Cmd+K → Ask → Agent — 위임 권한을 사람이 단계적으로 통제.



맥락 vs 실행 Rules는 ‘맥락’을 형성, Hooks는 ‘실행 동작’을 통제 — 역할이 다르다.



재사용·위임 삼형제 Commands(수동) · Skills(자동+스크립트) · Subagents(격리·병렬).



MCP 외부 도구를 표준 프로토콜로 연결 — ‘AI의 손발’ 확장. 보안 주의.



Hooks 실행 루프 전후에 스크립트 — 차단·마스킹·감사·자동 주입. 생성/검증 분리.

4

Cursor로 테트리스 만들고 GitHub에 배포하기

7~8교시 · 1~3장의 종합 적용 — 프롬프트만으로 동작하는 게임을 완주

이 장에서 다루는 것

- 준비: 프로젝트 폴더와 Cursor 열기
- 보드·블록 → 낙하·충돌·조작 → 줄 삭제·점수·게임 오버
- 실행과 테스트 (시나리오 체크리스트)
- Git으로 기록하고 GitHub에 올리기
- GitHub Pages로 누구나 접속하는 링크 배포
- 회고와 확장 과제

테트리스 — 단계별 프롬프트 흐름

각 ‘프롬프트’ 상자를 그대로 복사해 Cursor 채팅(Agent 모드)에 붙여 넣고, 결과(diff)를 검토·수락합니다. 한 번에 완벽하지 않아도 좋습니다 — 2장의 ‘반복 개발’을 그대로 적용합니다.



P0

프로젝트 골격

순수 HTML/CSS/JS로
index.html·style.css·script.js. 빈 보드 +
점수 영역만.



P1

보드·블록 렌더링

10×20 그리드, 7가지 테트로미노 정의,
draw()로 블록 하나 ‘그리기만’.



P2

자동 낙하·충돌

0.5초마다 한 칸 낙하, 바닥/블록에 닿으면 고정
후 새 블록 등장.



P3

키보드 조작

←→ 이동, ↓ soft drop, ↑ 회전, Space
hard drop. 회전 막히면 취소.



P4

라인 클리어·점수

가득 찬 줄 삭제·정렬, 다중 삭제 처리(1/2/3/4
줄=100/300/500/800).



P5

게임 오버·재시작

등장 공간 없으면 게임 오버, ‘다시 시작’ 버튼으
로 초기화.

프롬프트 6(다듬기) · 디버깅 프롬프트: ‘다음 블록 미리보기·레벨·반응형’, 증상을 구체적으로 적을수록 잘 고친다(2장 ‘실패 분석’).

실행 · 테스트 · Git · GitHub Pages 배포



실행과 테스트

index.html 더블클릭, 또는 로컬 서버(권장)로 실행.

```
python3 -m http.server 8000 # localhost:8000
```



Git으로 기록 (방법 A · 권장)

repo=버전 관리 단위 · commit=변경 스냅샷 · push=GitHub로 전송.

```
gh repo create tetris-cursor --public --source=. --push
```



GitHub Pages로 배포 — 누구나 접속하는 링크

저장소 → Settings → Pages → 'Deploy from a branch', main / 루트(/root) → Save. 잠시 후 <https://<사용자명>.github.io/tetris-cursor/> 에서 게임이 열린다.



테스트 시나리오 점검 (프롬프트 7)

① 7종 블록이 모두 나오는가 ② 좌우 이동·회전·하강이 되는가 ③ 줄이 차면 삭제·점수 ④ 4줄 동시 삭제=800점 ⑤ 게임 오버 후 재시작 정상 동작 — 안 되는 항목은 원인을 찾아 고친다.

회고와 확장 과제 · 4장 핵심 정리



회고

어떤 프롬프트에서 한 번에 성공했고 어디서 반복이 필요했나? 2장의 ‘실패 분석’이 어떻게 작동했나?



확장

사운드 효과 · 최고 점수 저장(localStorage) · 색상 테마 · 일시정지 기능 추가.



심화

4장 전체 과정을 하나의 Cursor Command(/deploy)나 Skill로 묶어 자동화 (3장 연계).

4장 핵심 정리

- 결과 중심 프롬프트를 ‘단계별’로 주면 비개발자도 동작하는 게임을 완성한다.
- 실행은 index.html 더블클릭 또는 로컬 서버. 테스트는 시나리오 체크리스트로.
- git init → commit → push로 버전을 기록·공유, GitHub Pages로 무료 배포.
- 이 장은 1~3장(리터러시·프롬프트·Cursor)의 종합 적용이다.

참고 자료

1장

- AI Futures Project, 'AI 2027' (ai-2027.com)
- Narayanan & Kapoor, 'AI as Normal Technology' (반대 관점)
- Vellum · LLM-Stats · Zapier — 2026 LLM 비교 자료

2장

- Andrew Ng & Isa Fulford, 'ChatGPT Prompt Engineering for Developers' (DeepLearning.AI × OpenAI)
- Stanford CS230 (2025) Lecture 8 — Agents, Prompts, and RAG
- Prompt Engineering Guide (promptingguide.ai)

3장

- Cursor 공식 문서 — cursor.com/docs (Agent · Rules · Skills · Subagents · MCP · Hooks)
- Cursor 체인지로그 — 1.7(Hooks), 2.4(Subagents · Skills)
- Agent Skills 개방형 표준 — agentskills.io

4장

- GitHub Docs — 저장소 생성 · Pages 배포 (docs.github.com)
- GitHub CLI — cli.github.com

저작권·시점 안내 — 본 교안은 위 자료들의 핵심을 강의자가 재구성·요약한 것으로 원문 표현을 그대로 옮기지 않았습니다. 모델 버전·제품 메뉴·기능명은 2026년 상반기 기준이며 빠르게 바뀌므로 강의 직전 공식 문서로 재확인하세요.



강의를 마치며

리터러시에서 배포까지, 하나의 흐름으로

개념을 의심하고, 프롬프트로 정교하게 부리고, 에이전트로 직접 만들어 세상에 배포하기 — 이 세 층위는 앞으로의 모든 작업에 그대로 적용됩니다.

Cursor AI를 활용한 소프트웨어 개발 기본 과정

Prepared by **DaeKyeong Kim, Ph.D.**